Lecture 14: Variational autoencoders (VAEs)

Last time we developed **black-box variational inference (BBVI)**: a general-purpose recipe for variational inference based on the score function gradient estimator. We saw that the main practical problem with vanilla BBVI is gradient variance, and that techniques like Rao-Blackwellization and control variates can help.

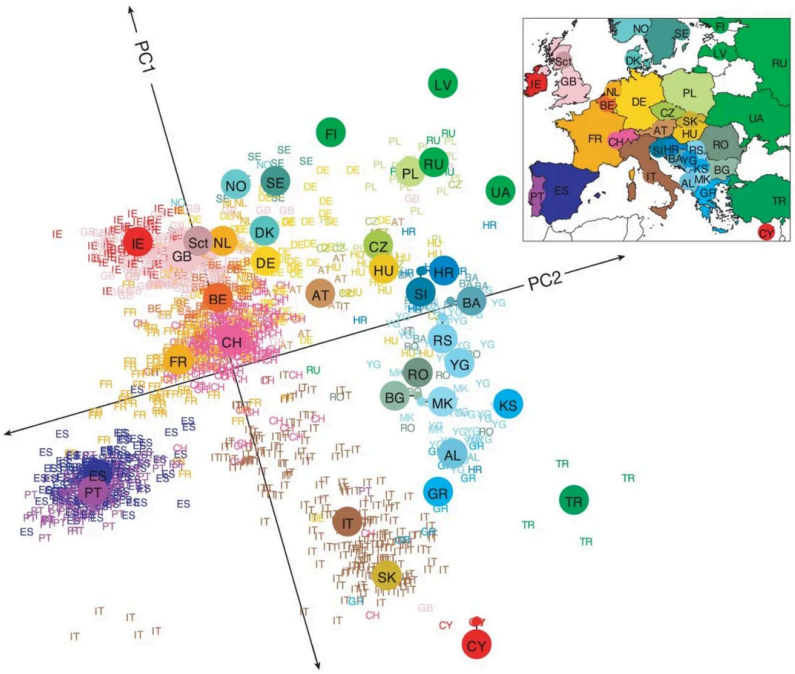
Today we develop an entirely different gradient estimator (i.e., **reparameterization gradient**) which often has dramatically lower variance, but which only applies when both the model and the variational distribution have a particular structure. The canonical example where it does apply is the **variational autoencoder (VAE)**.

Along the way we'll see that VAEs sit at an interesting intersection: they are simultaneously (a) a probabilistic version of an old idea from representation learning (autoencoders), (b) a deep generative model that scales to complex data like images, and (c) a scalable variational inference algorithm. The goal today is to develop each of these views.

PCA as a single-layer autoencoder

Recall **principal components analysis (PCA)**. Given data $\mathbf{x}_i \in \mathbb{R}^D$, we want to find low-dimensional representations $\mathbf{z}_i \in \mathbb{R}^K$ for $K \ll D$ that captures most of the variation in the data.

The famous example I mentioned in class is that of Novembre et al. (2009) "Genes mirror geography within Europe" where they applied PCA to an individuals-by-genes matrix and found that the 2-dimensional $\mathbf{z}_i$ inferred for individuals i recovered the map of Europe!



One way to define PCA modeling objective is to say that the model's "reconstruction" of $\mathbf{x}_i$ is $\hat{\mathbf{x}}_i = W\mathbf{z}_i$ where $W \in \mathbb{R}^{D \times K}$ is some orthonormal matrix such that $W^\top W = I_K$. The learning problem is then:

$$\min_{\mathbf{z}_{1:n},\, W \text{ s.t. } W^\top W = I_K} \sum_{i=1}^{n} \|\mathbf{x}_i - W\mathbf{z}_i\|^2$$

The orthonormality of W and the linearity of the model implies that:

$$\hat{\mathbf{x}}_i = W\mathbf{z}_i \tag{1}$$
$$W^\top \hat{\mathbf{x}}_i = W^\top W \mathbf{z}_i \tag{2}$$
$$W^\top \hat{\mathbf{x}}_i = \mathbf{z}_i \tag{3}$$

Moreover, since a "good" model should roughly satisfy $\mathbf{x}_i \approx \hat{\mathbf{x}}_i$, we can plug-in $W^\top \mathbf{x}_i$ for $\mathbf{z}_i$ to obtain an objective function for only W:

$$\min_{W \text{ s.t. } W^\top W = I_K} \sum_{i=1}^{n} \|\mathbf{x}_i - WW^\top \mathbf{x}_i\|^2$$

This might seem like a funny objective. We are saying that we want to find some W matrix such that $\mathbf{x}_i \approx WW^\top \mathbf{x}_i$. Why would we do such a thing?

As it turns out, this is an instance of an **autoencoder**. (The prefix "auto" means "self" in Greek.) We are learning a linear function $A \in \mathbb{R}^{D \times D}$ which maps $\mathbf{x}_i \in \mathbb{R}^D$ to a reconstruction of itself $A\mathbf{x}_i = \hat{\mathbf{x}}_i$. If we could learn *any* $D \times D$ linear map, then this learning objective would achieve nothing (we

would just learn the identity function $A = I_D$). However, the linear map we seek to learn is *constrained* $A = WW^\top$. We are thus creating an **information bottleneck**: the model must learn how to reconstruct $\mathbf{x}_i$ using only a subset of the information in $\mathbf{x}_i$. In so doing, it must learn "important" or "high-level" features of the data.

More generally, define the following two functions:

- the **encoder** $\mathbf{z} = g_\phi(\mathbf{x})$
- the **decoder** $\hat{\mathbf{x}} = f_\theta(\mathbf{z})$

In general, the **autoencoding objective** for some loss function $\ell(\cdots)$ is:

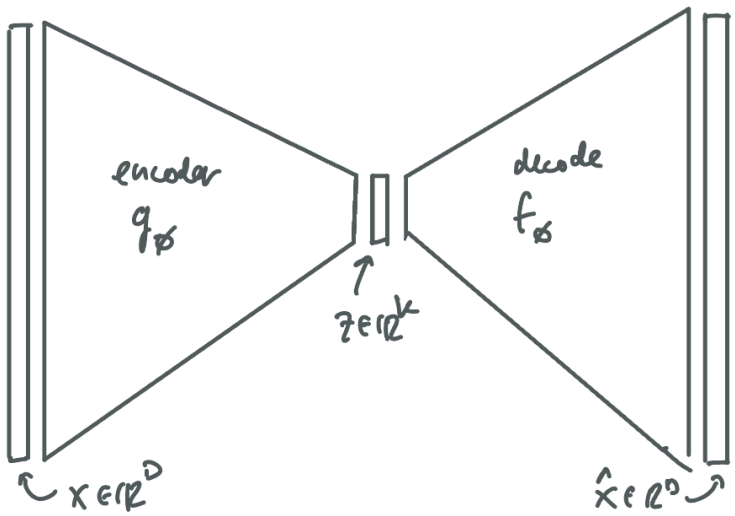
$$\min_{\theta,\phi} \sum_{i=1}^{n} \ell\big(\mathbf{x}_i,\ f_\theta(g_\phi(\mathbf{x}_i))\big)$$

If our functions g_ϕ and f_θ are unconstrained, this objective is trivially easy to satisfy. However, in the case of PCA, they are highly constrained:

- the **decoder** $f_\theta(\mathbf{z}) \equiv W\mathbf{z}$
- the **encoder** $g_\phi(\mathbf{x}) \equiv W^\top\mathbf{x}$

So PCA is a **linear autoencoder with tied weights**. One can show that when the loss is the squared loss, the minimizer then recovers the top K principal components.

More generally though, PCA is an autoencoder that introduces an "information bottleneck" explicitly since $g_\phi : \mathbb{R}^D \to \mathbb{R}^K$ and $f_\theta : \mathbb{R}^K \to \mathbb{R}^D$ for $K \ll D$.

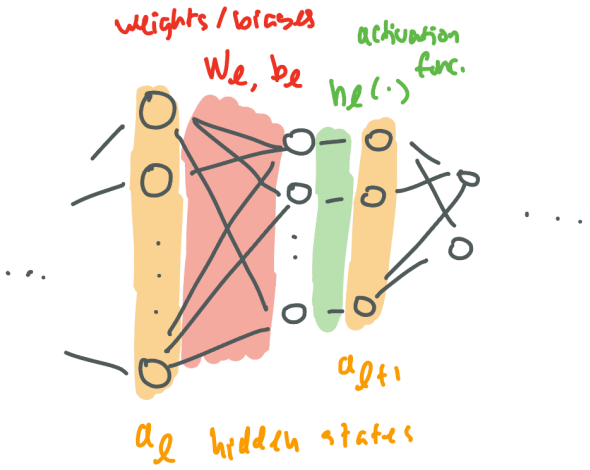


The natural generalization is then to replace the single-layer/linear encoder and decoder with much more flexible functions, such as **neural networks**.

A brief primer on neural networks

A **feed-forward neural network** is a parametric function $f_\theta : \mathbb{R}^{D_{\text{in}}} \to \mathbb{R}^{D_{\text{out}}}$ built by composing affine maps with element-wise non-linearities:

$$\mathbf{a}_0 = \mathbf{x}, \qquad \mathbf{a}_\ell = h_\ell(W_\ell\, \mathbf{a}_{\ell-1} + \mathbf{b}_\ell) \quad \text{for } \ell = 1, \dots, L, \qquad f_\theta(\mathbf{x}) = \mathbf{a}_L$$



The **parameters** $\theta = (W_1, \mathbf{b}_1, \dots, W_L, \mathbf{b}_L)$ are the weights and biases at each layer. The function h_ℓ is an **activation function** applied element-wise — e.g., ReLU $h(t) = \max(0, t)$, tanh, or sigmoid. Each $\mathbf{a}_\ell$ is called a **hidden representation** at layer ℓ.

Why the non-linearities? Suppose we left them out — i.e., set h_ℓ to the identity at every layer. Then the whole network collapses to a single affine map:

$$f_\theta(\mathbf{x}) \;=\; W_L \cdots W_2 W_1\, \mathbf{x} \;+\; (\text{accumulated bias}) \;=\; W\mathbf{x} + \mathbf{b}$$

The composition of linear maps is linear. Thus no matter how many layers you stack, the function class is still just $\{\mathbf{x} \mapsto W\mathbf{x} + \mathbf{b}\}$. So, congrats, you made a "deep neural network" but are **still just doing linear regression**. The non-linearities are thus what make deep neural networks expressive function classes.

Denoising autoencoders

If we allow f_θ and g_ϕ to be flexible enough, then we will learn them such that $g_\phi \circ f_\theta$ is the identity. In other words, the autoencoder learns to just repeat back the data and thus does not learn any useful representation.

One classical fix is the **denoising autoencoder**. In this setting, we corrupt the input with noise and ask the autoencoder to reconstruct the *clean* version:

$$\tilde{\mathbf{x}}_i \sim e(\tilde{\mathbf{x}}_i \mid \mathbf{x}_i), \quad \mathbf{z}_i = f_\theta(\tilde{\mathbf{x}}_i), \quad \hat{\mathbf{x}}_i = g_\phi(\mathbf{z}_i), \quad \min_{\theta,\phi} \sum_{i=1}^{n} \ell(\mathbf{x}_i - g_\phi(f_\theta(\tilde{\mathbf{x}}_i)))$$

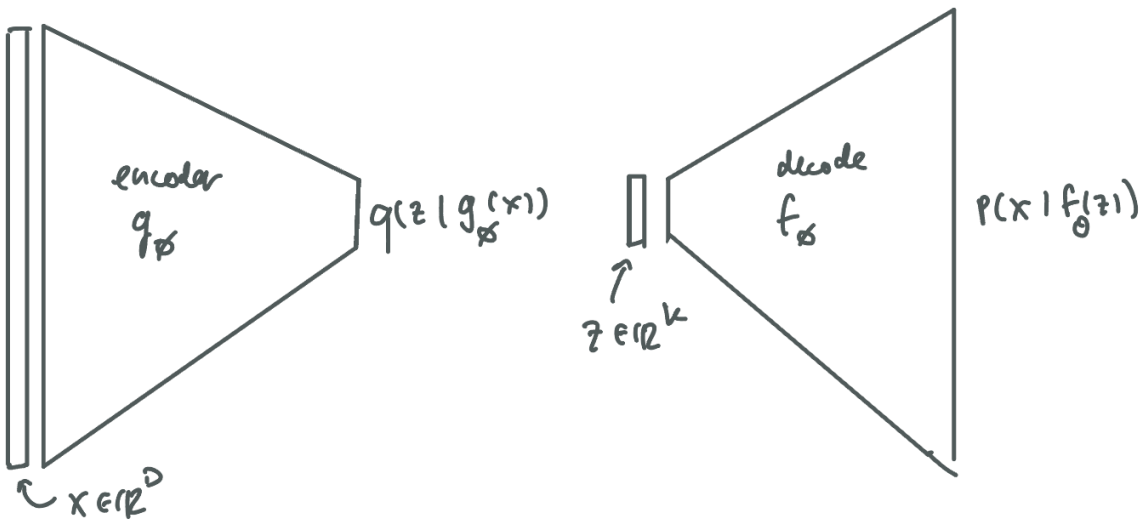
Thus, the autoencoder must learn to "undo" the noise. The encoder cannot simply copy its input: it must learn higher-level features that are mostly invariant to noise.

These ideas come from representation learning, but they are not probabilistic. The encoder is a deterministic function, there is no posterior distribution, and there is no model from which we could sample new $\mathbf{x}$.

Variational autoencoders (VAEs)

The **variational autoencoder** (Kingma and Welling, 2013; Rezende et al., 2014) is the probabilistic version of an autoencoder. Instead of the encoder and decoder networks outputting $\mathbf{z}$ and $\mathbf{x}$ directly, they output *the parameters of distributions* over $\mathbf{x}$ and $\mathbf{z}$. For the standard/vanilla VAE, these distributions are Gaussian.

- **the decoder** $p_\theta(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}(\mathbf{x} \mid \mu_\theta(\mathbf{z}), \Sigma_\theta(\mathbf{z}))$
  - input is latent code $\mathbf{z} \in \mathbb{R}^K$
  - network returns parameters $f_\theta(\mathbf{z}) \equiv [\mu_\theta(\mathbf{z}), \Sigma_\theta(\mathbf{z})]$
  - output is a sample $\mathbf{x} \sim \mathcal{N}(\mathbf{x} \mid \mu_\theta(\mathbf{z}), \Sigma_\theta(\mathbf{z}))$
- **the encoder** $q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\mathbf{x} \mid \mu_\phi(\mathbf{x}), \Sigma_\phi(\mathbf{x}))$
  - input is an observation $\mathbf{x} \in \mathbb{R}^D$
  - network returns parameters $g_\phi(\mathbf{z}) \equiv [\mu_\phi(\mathbf{x}), \Sigma_\phi(\mathbf{x})]$
  - output is a sample $\mathbf{z} \sim \mathcal{N}(\mathbf{z} \mid \mu_\theta(\mathbf{x}), \Sigma_\theta(\mathbf{x}))$



What's the point of this? Well say that $\mathbf{x}$ is very high-dimensional or complex (e.g., an image) and notice that the decoder defines a full probability distribution over $\mathbf{x}$. Then if we introduce a simple prior over the code $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, I_K)$, we could in principle generate new synthetic $\mathbf{x}$'s as follows:

$$\mathbf{z} \sim \mathcal{N}(\mathbf{0}, I_K) \tag{4}$$
$$\mathbf{x} \sim \mathcal{N}(\mathbf{x} \mid \mu_\theta(\mathbf{z}), \Sigma_\theta(\mathbf{z})) \tag{5}$$

where notice that we can now view p_θ as defining a full joint distribution:

$$p_\theta(\mathbf{x}, \mathbf{z}) = \mathcal{N}(\mathbf{z} \mid \mathbf{0}, I_K)\,\mathcal{N}(\mathbf{x} \mid \mu_\theta(\mathbf{z}), \Sigma_\theta(\mathbf{z}))$$

Specifying (realistic) **generative models** for complex data (e.g., images) is generally difficult to do manually, so this would be nice if it worked. However, this all leaves aside the question of how to actually fit p_θ.

Training VAEs: Variational EM

Forgetting the encoder for a second, how would we go about training the decoder using only the tools we have discussed so far in class? Say we have a batch of images $\mathbf{x}_{1:n}$. The natural first step is to write down the maximum marginal likelihood objective:

$$\max_\theta \sum_{i=1}^n \log p_\theta(\mathbf{x}_i)$$

where $\log p_\theta(\mathbf{x}_i) = \log \int p_\theta(\mathbf{x}_i \mid \mathbf{z}_i) p_\theta(\mathbf{z}_i) d\mathbf{z}_i$. However, as usual, the marginal likelihood is (very, very) intractable. So we might then reach for EM:

$$\max_q \mathcal{L}(\theta, q) \equiv \max_q \mathbb{E}_{q(\mathbf{z}_{1:n})}\left[\log \frac{p_\theta(\mathbf{x}_{1:n}, \mathbf{z}_{1:n})}{q(\mathbf{z}_{1:n})}\right] \quad \text{(E-step)} \tag{6}$$
$$\max_\theta \mathcal{L}(\theta, q) \equiv \max_q \mathbb{E}_{q(\mathbf{z}_{1:n})}\left[\log p_\theta(\mathbf{x}_{1:n}, \mathbf{z}_{1:n})\right] \quad \text{(M-step)} \tag{7}$$

Think about $q(\mathbf{z}_{1:n})$. As we know, the optimal E-step is one that sets:

$$q^\star(\mathbf{z}_{1:n}) = \arg\max_q \mathcal{L}(\theta, q) = p_\theta(\mathbf{z}_{1:n} \mid \mathbf{x}_{1:n})$$

For simple models, this is tractable. For complex models (like this one) it is not. An alternative is what is called **variational EM** where we constrain q to be in some parametric family (e.g., the factorized family):

$$q(\mathbf{z}_{1:n}) \equiv q_\phi(\mathbf{z}_{1:n}) = \prod_{i=1}^n q(\mathbf{z}_i \mid \phi_i)$$

Moreover, **generalized EM** is a further variation that replaces both maximization steps with gradient updates:

$$\phi_t \leftarrow \phi_{t-1} + \rho_t \nabla_\phi \, \mathbb{E}_{q_\phi(\mathbf{z}_{1:n})} \left[\log \frac{p_{\theta_{t-1}}(\mathbf{x}_{1:n}, \mathbf{z}_{1:n})}{q_\phi(\mathbf{z}_{1:n})} \right] \quad \text{(E-step)} \tag{8}$$

$$\theta_t \leftarrow \theta_{t-1} + \rho_t \nabla_\theta \, \mathbb{E}_{q_{\phi_{t-1}}(\mathbf{z}_{1:n})} \left[\log p_\theta(\mathbf{x}_{1:n}, \mathbf{z}_{1:n}) \right] \quad \text{(M-step)} \tag{9}$$

(Generalized EM with an exact E-step is still guaranteed to converge to a local optimum of the marginal likelihood with suitable step-sizes. When we introduce a variational E-step, it is now only guaranteed to converge to a local optimum of the ELBO, though this is still often good enough in many applications.)

We haven't yet said how we can evaluate/estimate these gradients, but trust me (for now) that we can (**spoiler alert**: reparameterization trick + autodiff). We will first focus on a different aspect of the problem.

Training VAEs: Amortized variational inference

Even if we can take/estimate the gradients above, we should address a different problem. Say n is very large (as it should be if we want to learn a generative model of images). Our variational distribution above dedicates one variational parameter ϕ_i for every data point. That's a lot of parameters. Too many.

What if instead, we had a neural network that predicted ϕ_i from $\mathbf{x}_i$:

$$\phi_i \approx g_\phi(\mathbf{x}_i)$$

Here I am slightly overloading abusing notation: ϕ_i is the optimal variational parameter for $\mathbf{x}_i$, while g_ϕ is a neural network whose parameters (i.e., weights and biases) are denoted ϕ.

Our variational family can now be written:

$$q_\phi(\mathbf{z}_{1:n} \mid \mathbf{x}_{1:n}) = \prod_{i=1}^n q_\phi(\mathbf{z} \mid \mathbf{x}_i) = \prod_{i=1}^n q(\mathbf{z}_i \mid g_\phi(\mathbf{x}_i))$$

where notice that the variational parameters ϕ are no longer growing with the data. This is called **amortization**: we are "amortizing" the cost of the variational E-step by learning the parameters of a shared flexible function that maps the data to the variational parameters. And so voila, the **variational "encoder"** is born.

Training VAEs: High-level overview

To summarize, for model and variational parameters θ and ϕ respectively, which are the weights/biases of the encoder g_ϕ and decoder f_θ networks, we will seek to maximize the ELBO:

$$\mathcal{L}(\theta, \phi) = \sum_{i=1}^{n} \mathbb{E}_{\mathbf{z}_i \sim q_\phi(\mathbf{z}_i|\mathbf{x}_i)} \left[\log \frac{p_\theta(\mathbf{z}_i, \mathbf{x}_i)}{q_\phi(\mathbf{z}_i \mid \mathbf{x}_i)} \right] \tag{10}$$

$$\equiv \sum_{i=1}^{n} \mathbb{E}_{\mathbf{z}_i \sim q(\mathbf{z}_i|g_\phi(\mathbf{x}_i))} \left[\log \frac{p(\mathbf{x}_i \mid f_\theta(\mathbf{z}_i))\, p_\theta(\mathbf{z}_i)}{q(\mathbf{z}_i \mid g_\phi(\mathbf{x}_i))} \right] \tag{11}$$

As always, we can perform "ELBO surgery" to further understand this objective:

$$\mathcal{L}(\theta, \phi) = \sum_{i=1}^{n} \mathbb{E}_{\mathbf{z}_i \sim q_\phi(\mathbf{z}_i|\mathbf{x}_i)} [\log p(\mathbf{x}_i \mid f_\theta(\mathbf{z}_i))] + \mathrm{KL}\left(q_\phi(\mathbf{z}_i \mid \mathbf{x}_i) \mid\mid p_\theta(\mathbf{z}_i)\right)$$

Notice the structure:

- The first term is the expected **reconstruction log-likelihood**: how well does the decoder reconstruct $\mathbf{x}_i$ from $\mathbf{z}_i \sim q_\phi(\mathbf{z}_i \mid \mathbf{x}_i)$?
- The second term is a **KL regularizer** pulling $q_\phi(\mathbf{z} \mid \mathbf{x})$ toward the prior $p_\theta(\mathbf{z})$.

Deep Gaussian generative models

As mentioned earlier, the vanilla/standard VAE adopts Gaussian assumptions for the prior, likelihood and variational family. This is idea is based on deep Gaussian generative models. Consider an L-layer **hierarchy of Gaussian draws**, where the data lives at the bottom ($\mathbf{z}_0 \equiv \mathbf{x}$) and the topmost latent ($\mathbf{z}_L$) is a standard Gaussian. Each layer is centered around a neural network applied to the layer above, with *both* its mean and covariance produced by neural networks of that layer:

$$\mathbf{z}_L \sim \mathcal{N}(\mathbf{0}, I_K) \tag{12}$$

$$\mathbf{z}_{L-1} \sim \mathcal{N}\big(\mu_{L-1}(\mathbf{z}_L),\, \Sigma_{L-1}(\mathbf{z}_L)\big) \tag{13}$$

$$\vdots \tag{14}$$

$$\vdots \tag{15}$$

$$\mathbf{z}_1 \sim \mathcal{N}\big(\mu_1(\mathbf{z}_2),\, \Sigma_1(\mathbf{z}_2)\big) \tag{16}$$

$$\mathbf{x} \equiv \mathbf{z}_0 \sim \mathcal{N}\big(\mu_0(\mathbf{z}_1),\, \Sigma_0(\mathbf{z}_1)\big)$$

Each μ_ℓ and Σ_ℓ is its own neural network, with its own parameters. This is clearly a very flexible class of generative models. This *looks* much richer than the single-layer VAE decoder $\mathbf{x} \sim \mathcal{N}(f_\theta(\mathbf{z}), \Sigma_\theta(\mathbf{z}))$ but it actually isn't. To see this, we will use the **reparameterization of a**

Gaussian: any draw $\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$ can equivalently be written as a deterministic affine transformation of *white noise*:

$$\mathbf{z} \;=\; \boldsymbol{\mu} \;+\; \Sigma^{1/2}\,\boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, I)$$

where $\Sigma^{1/2}$ is any matrix square root of Σ (e.g., its Cholesky factor) satisfying $\Sigma^{1/2}(\Sigma^{1/2})^{\top} = \Sigma$.

Reparameterizing each layer. Apply this fact at every step of the hierarchy:

$$\begin{aligned}
\mathbf{z}_L &= \boldsymbol{\varepsilon}_L && (17)\\
\mathbf{z}_{L-1} &= \mu_{L-1}(\mathbf{z}_L) \;+\; \Sigma_{L-1}^{1/2}(\mathbf{z}_L)\,\boldsymbol{\varepsilon}_{L-1} && (18)\\
&\;\;\vdots && (19)\\
\mathbf{z}_1 &= \mu_1(\mathbf{z}_2) \;+\; \Sigma_1^{1/2}(\mathbf{z}_2)\,\boldsymbol{\varepsilon}_1 && (20)\\
\mathbf{x} &= \mu_0(\mathbf{z}_1) \;+\; \Sigma_0^{1/2}(\mathbf{z}_1)\,\boldsymbol{\varepsilon}_0 && (21)
\end{aligned}$$

where each $\boldsymbol{\varepsilon}_\ell \overset{iid}{\sim} \mathcal{N}(\mathbf{0}, I)$. Note there are no more Gaussian *draws* interleaved in the hierarchy. We could sample all the $\boldsymbol{\varepsilon}_\ell$ first for all layers, and compute $\mathbf{x}$ deterministically. To see this, first define the function $\mathbf{z}_\ell(\mathbf{z}_{\ell+1},\, \boldsymbol{\varepsilon}_\ell)$ to be the function of the previous layer and noise:

$$\mathbf{z}_\ell(\mathbf{z}_{\ell+1},\, \boldsymbol{\varepsilon}_\ell) \;\equiv\; \mu_\ell(\mathbf{z}_{\ell+1}) \;+\; \Sigma_\ell^{1/2}(\mathbf{z}_{\ell+1})\,\boldsymbol{\varepsilon}_\ell$$

Then substitute each equation into the next, working from the top down. Ultimately we have that $\mathbf{x}$ is a deterministic function of *all* the noise vectors $(\boldsymbol{\varepsilon}_0, \boldsymbol{\varepsilon}_1, \ldots, \boldsymbol{\varepsilon}_L)$:

$$\mathbf{x} \;=\; \mathbf{z}_0\Big(\mathbf{z}_1\big(\cdots \mathbf{z}_{L-1}(\boldsymbol{\varepsilon}_L,\, \boldsymbol{\varepsilon}_{L-1}) \cdots ,\, \boldsymbol{\varepsilon}_1\big),\, \boldsymbol{\varepsilon}_0\Big)$$

Concatenated into a single big vector, they are all still multivariate Gaussian:

$$\boldsymbol{\varepsilon} \;\equiv\; \big[\boldsymbol{\varepsilon}_0;\, \boldsymbol{\varepsilon}_1;\, \ldots;\, \boldsymbol{\varepsilon}_L\big] \;\sim\; \mathcal{N}(\mathbf{0}, I)$$

Thus we can simply rewrite the composition of functions as f_θ, and the whole hierarchy collapses to:

$$\mathbf{x} \;=\; \mathbf{x}_\theta(\boldsymbol{\varepsilon}), \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, I)$$

where $\mathbf{x}_\theta$ is the single (deep) neural network formed by composing the per-layer μ_ℓ and $\Sigma_\ell^{1/2}$ neural networks and θ are the union of the per-layer parameters. Or equivalently:

$$\mathbf{x} \;=\; \mu_\theta(\boldsymbol{\varepsilon}) + \Sigma_\theta^{1/2}(\boldsymbol{\varepsilon})\boldsymbol{\varepsilon}_0$$

Reparameterization gradients

Let's turn back to how to train VAEs using generalized/variational EM. Above we stated the procedure. Consider the variational/generalized E-step

$$\phi_t \leftarrow \phi_{t-1} + \rho_t \nabla_\phi \, \mathbb{E}_{q_\phi}\left[\log \frac{p_\theta(\mathbf{x}_{1:n}, \mathbf{z}_{1:n})}{q_\phi(\mathbf{z}_{1:n} \mid \mathbf{x}_{1:n})}\right] \quad \text{(E-step)} \tag{22}$$

As we saw from last time (BBVI) this is a "stochastic gradient", in the sense that the parameter ϕ governs the expectation. Last time we used the "score function gradient estimator" to evaluate this gradient. The score function trick told us:

$$\nabla_\phi \mathcal{L}(\phi, \theta) \;=\; \nabla_\phi \mathbb{E}_{\mathbf{z}_{1:n} \sim q_\phi}\left[\log \frac{p_\theta(\mathbf{x}_{1:n}, \mathbf{z}_{1:n})}{q_\phi(\mathbf{z}_{1:n} \mid \mathbf{x}_{1:n})}\right] \;=\; \sum_{i=1}^n \mathbb{E}_{\mathbf{z}_i \sim q_\phi}\Big[R_{\theta,\phi}(\mathbf{x}_i, \mathbf{z}_i) \, \nabla_\phi \log q_\phi(\mathbf{z}_i \mid \mathbf{x}_i)\Big]$$

where the **learning signal (or "reward")** is defined as $R_{\theta,\phi}(\mathbf{x}_i, \mathbf{z}_i) \triangleq \log \frac{p_\theta(\mathbf{x}_i, \mathbf{z}_i)}{q_\phi(\mathbf{z}_i | \mathbf{x}_i)}$.

Thus we could obtain an unbiased estimate of the gradient as with $i \sim \text{Uniform}(1, \dots, n)$ and $\mathbf{z}_i \sim q_\phi(\mathbf{z}_i \mid \mathbf{x}_i)$:

$$\widehat{\nabla}_\phi \mathcal{L}(\phi, \theta) \;=\; n R_{\theta,\phi}(\mathbf{x}_i, \mathbf{z}_i) \, \nabla_\phi \log q_\phi(\mathbf{z}_i)$$

This is unbiased but typically suffers from high variance. For standard VAEs we will use a different trick which much nicer properties.

Recall that $q_\phi(\mathbf{z}_i \mid \mathbf{x}_i) = \mathcal{N}(\mathbf{z}_i \mid \mu_\phi(\mathbf{x}_i), \Sigma_\phi(\mathbf{x}_i))$ Using the reparameterization of the Gaussian that we saw above, we can rewrite

$$\mathbf{z}_i \sim \mathcal{N}(\mu_\phi(\mathbf{x}_i), \Sigma_\phi(\mathbf{x}_i)) \iff \mathbf{z}_i \triangleq \mathbf{z}_\phi(\boldsymbol{\varepsilon}_i, \mathbf{x}_i) = \mu_\phi(\mathbf{x}_i) + \Sigma_\phi(\mathbf{x}_i)^{1/2} \boldsymbol{\varepsilon}_i \text{ where } \boldsymbol{\varepsilon}_i \sim \mathcal{N}(0, I)$$

So the ELBO can be rewritten as an expectation with respect to white noise:

$$\mathcal{L}(\phi, \theta) = \sum_{i=1}^n \mathbb{E}_{\mathbf{z}_i \sim q_\phi}\left[\log \frac{p_\theta(\mathbf{x}_i, \mathbf{z}_i)}{q_\phi(\mathbf{z}_i \mid \mathbf{x}_i)}\right] = \sum_{i=1}^n \mathbb{E}_{\boldsymbol{\varepsilon}_i \sim \mathcal{N}(0,I)}\left[\log \frac{p_\theta(\mathbf{x}_i, \mathbf{z}_\phi(\boldsymbol{\varepsilon}_i, \mathbf{x}_i))}{q_\phi(\mathbf{z}_\phi(\boldsymbol{\varepsilon}_i, \mathbf{x}_i) \mid \mathbf{x}_i)}\right]$$

Now when we take a gradient with respect to ϕ, it pushes past the expectation, as the distribution governing the expectation **no longer depends on** ϕ:

$$\nabla_\phi \mathcal{L}(\phi, \theta) = \sum_{i=1}^n \nabla_\phi \mathbb{E}_{\mathbf{z}_i \sim q_\phi}\Big[R_{\theta,\phi}(\mathbf{x}_i, \mathbf{z}_i)\Big] = \sum_{i=1}^n \mathbb{E}_{\boldsymbol{\varepsilon}_i \sim \mathcal{N}(0,I)}\Big[\nabla_\phi R_{\theta,\phi}(\mathbf{x}_i, \mathbf{z}_\phi(\boldsymbol{\varepsilon}_i, \mathbf{x}_i))\Big]$$

This is the **"reparameterization trick"** and it leads to the following unbiased gradient estimator with $i \sim \mathrm{Uniform}(1, \ldots, n)$ and $\boldsymbol{\varepsilon}_i \sim \mathcal{N}(0, I)$:

$$\widehat{\nabla}_\phi \mathcal{L}(\phi, \theta) \;=\; n \nabla_\phi R_{\theta,\phi}(\mathbf{x}_i, \mathbf{z}_\phi(\boldsymbol{\varepsilon}_i, \mathbf{x}_i))$$

This is the **reparameterization gradient estimator**. Notice that the gradient now flows *through* the learning signal $R_{\theta,\phi}(\mathbf{x}_i, \mathbf{z}_\phi(\boldsymbol{\varepsilon}_i, \mathbf{x}_i))$. This differentiates the reward signal, rather than multiplying the reward signal by the score function. It turns out that this leads to much lower gradient variance.

Automatic differentiation

How do we compute the gradient of the reward signal $\nabla_\phi R_{\theta,\phi}$? Recall that here $\phi = (W_1, \mathbf{b}_1, \ldots, W_L, \mathbf{b}_L)$ are the weights and biases of the encoder network. The math is "just" the chain rule. Consider the gradient w.r.t. one weight matrix W_ℓ: the loss $\mathcal{L}$ depends on $\mathbf{a}_L$, which depends on $\mathbf{a}_{L-1}$, ..., which depends on $\mathbf{a}_\ell$, which depends on W_ℓ. Stringing it all together:

$$\nabla_{W_\ell} \mathcal{L} \;=\; \frac{\partial \mathcal{L}}{\partial \mathbf{a}_L} \cdot \frac{\partial \mathbf{a}_L}{\partial \mathbf{a}_{L-1}} \cdot \frac{\partial \mathbf{a}_{L-1}}{\partial \mathbf{a}_{L-2}} \cdots \frac{\partial \mathbf{a}_{\ell+1}}{\partial \mathbf{a}_\ell} \cdot \frac{\partial \mathbf{a}_\ell}{\partial W_\ell}$$

The chain rule gives us a **symbolic** representation of the answer, but does not tell us the best way to **evaluate** it (i.e., compute). More specifically, there are different orders in which could evaluate the intermediate Jacobians. The two natural choices correspond to the two modes of autodiff: forward-mode and backward-mode.

- **Forward-mode: evaluate from the parameter end.** Pick a parameter W_ℓ; the running intermediate after k steps outward is $\partial \mathbf{a}_{\ell+k} / \partial W_\ell$ — how layer $\ell + k$'s activations depend on the parameter:

$$\frac{\partial \mathbf{a}_\ell}{\partial W_\ell} \;\to\; \frac{\partial \mathbf{a}_{\ell+1}}{\partial W_\ell} \;\to\; \cdots \;\to\; \frac{\partial \mathbf{a}_L}{\partial W_\ell} \;\to\; \frac{\partial \mathcal{L}}{\partial W_\ell}$$

Equivalent to perturbing W_ℓ and propagating the perturbation forward through the network. **One sweep yields the gradient for one parameter.** Getting every parameter's gradient requires one sweep *per parameter*:

$$\text{cost} \;\approx\; \#\text{params} \;\times\; \text{cost of one forward pass}$$

Infeasible for any realistic network.

- **Reverse-mode (a.k.a. backpropagation): evaluate from the loss end.** Do one forward pass, *caching every intermediate activation*; then walk backward through the network, maintaining a

running "gradient slot" at each layer — the gradient of the loss w.r.t. that layer's activations. The running intermediate steps backward through the network:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}_L} \to \frac{\partial \mathcal{L}}{\partial \mathbf{a}_{L-1}} \to \cdots \to \frac{\partial \mathcal{L}}{\partial \mathbf{a}_\ell} \to \frac{\partial \mathcal{L}}{\partial W_\ell}$$

Each step applies one layer's Jacobian to the running gradient, using the cached activations to evaluate it. Crucially, **a single backward sweep yields the gradient for *every* parameter at once** — because the chain's "anchor" is the (one) scalar loss, not any one parameter:

$$\text{cost} \;\approx\; \text{cost of one forward pass}$$

This is the algorithmic innovation that makes deep learning practical. Computing the gradient of a scalar loss with respect to *millions* of parameters costs essentially the same as evaluating the loss a single time.

In conclusion: backprop is **NOT** just the chain rule!!.

Bonus: the bits-back coding argument

Say you want to send messages $\mathbf{x}_i$ and encode them such that the message length is minimized. We know from the **source coding theorem** that the best possible coding scheme has message lengths

$$\ell\big(c^*(\mathbf{x}_i)\big) \;=\; \log_2 \tfrac{1}{p(\mathbf{x}_i)} \qquad (\text{optimal coding scheme})$$

What if $p(\mathbf{x})$ is unknown and intractable? For instance, what if $\mathbf{x}$ is an image from the set of all images? Let's instead represent it as a **mixture distribution** of tractable components:

$$p(\mathbf{x}_i) \;=\; \int p_\theta(\mathbf{x}_i \mid \mathbf{z}_i)\, p_\theta(\mathbf{z}_i)$$

For a given $\mathbf{z}_i$, we can code $\mathbf{x}_i$ according to $p_\theta(\mathbf{x}_i \mid \mathbf{z}_i)$. The length of that code is

$$\ell\big(c(\mathbf{x}_i \mid \mathbf{z}_i)\big) \;=\; \log_2 \tfrac{1}{p_\theta(\mathbf{x}_i|\mathbf{z}_i)}$$

Assume the sender and receiver both have access to $p_\theta(\mathbf{x}, \mathbf{z})$, so the receiver can decode $\mathbf{x}$. The sender will have to transmit the $\mathbf{z}_i$ they used to encode $\mathbf{x}_i$ so that the receiver can decode $\mathbf{x}_i$. Sending $\mathbf{z}_i$ will cost extra bits. How many?

$$\underbrace{\log_2 \tfrac{1}{p_\theta(\mathbf{z}_i)}}_{\text{cost to encode } \mathbf{z}_i} \;+\; \underbrace{\log_2 \tfrac{1}{p_\theta(\mathbf{x}_i|\mathbf{z}_i)}}_{\text{cost to encode } \mathbf{x}_i|\mathbf{z}_i}$$

Which $\mathbf{z}_i$ should the sender use to encode $\mathbf{x}_i$? One possibility would be

$$\mathbf{z}_i \;=\; \arg\max_{\mathbf{z}_i} \; p_\theta(\mathbf{x}_i \mid \mathbf{z}_i)$$

This would minimize the number of bits used to encode $\mathbf{x}_i \mid \mathbf{z}_i$. However it might then require many bits to encode $\mathbf{z}_i$, if the selected value has low probability under $p_\theta(\mathbf{z}_i)$.

Instead, what if the sender **sampled** $\mathbf{z}_i$ from the distribution $p_\theta(\mathbf{z}_i \mid \mathbf{x}_i)$? (Assume for now they can.)

$$\mathbf{z}_i \;\sim\; p_\theta(\mathbf{z}_i \mid \mathbf{x}_i)$$

Sampling from a distribution involves using **random bits**. This is a resource. The key idea with "bits-back" is that the receiver can **recover the random bits** that the sender used to sample $\mathbf{z}_i$.

Sender:

1. $\mathbf{z}_i \sim p_\theta(\mathbf{z}_i \mid \mathbf{x}_i)$
2. Encode $\mathbf{z}_i$ using $p_\theta(\mathbf{z}_i)$, encode $\mathbf{x}_i$ using $p_\theta(\mathbf{x}_i \mid \mathbf{z}_i)$.
3. Send $c(\mathbf{z}_i, \mathbf{x}_i)$.

Receiver:

1. Decode $\mathbf{z}_i$ using $p_\theta(\mathbf{z}_i)$, decode $\mathbf{x}_i$ using $p_\theta(\mathbf{x}_i \mid \mathbf{z}_i)$.
2. Recover random bits using $p_\theta(\mathbf{z}_i \mid \mathbf{x}_i)$.

So the total cost of the message, accounting for the bits-back refund, is:

$$\underbrace{\log_2 \tfrac{1}{p_\theta(\mathbf{z}_i)}}_{\text{cost to encode } \mathbf{z}_i} \;+\; \underbrace{\log_2 \tfrac{1}{p_\theta(\mathbf{x}_i|\mathbf{z}_i)}}_{\text{cost to encode } \mathbf{x}_i|\mathbf{z}_i} \;-\; \underbrace{\log_2 \tfrac{1}{p_\theta(\mathbf{z}_i|\mathbf{x}_i)}}_{\text{bits back}} \;=\; \underbrace{\log_2 \tfrac{1}{p_\theta(\mathbf{x}_i)}}_{\text{optimal rate!}}$$

So if the receiver recovers the random bits, and then uses them to encode their message back, we achieve the optimal rate of compression. This is the **bits-back argument**, which has been used to motivate latent variable models.

One problem with this argument is that if $p_\theta(\mathbf{x}_i)$ is intractable then so is $p_\theta(\mathbf{z}_i \mid \mathbf{x}_i)$. So we can't actually do what we just said. This is where we can introduce an **approximate posterior**:

$$q_\phi(\mathbf{z}_i \mid \mathbf{x}_i) \;\approx\; p_\theta(\mathbf{z}_i \mid \mathbf{x}_i)$$

We will want to fit the parameters ϕ so that q_ϕ is close to the exact (intractable) posterior. We will still encode messages according to the joint $p_\theta(\mathbf{z}_i, \mathbf{x}_i)$.

What's the total cost of a message using q_ϕ in place of the true posterior?

$$\underbrace{\log_2 \frac{1}{p_\theta(\mathbf{z}_i)}}_{\text{cost to encode } \mathbf{z}_i} + \underbrace{\log_2 \frac{1}{p_\theta(\mathbf{x}_i|\mathbf{z}_i)}}_{\text{cost to encode } \mathbf{x}_i|\mathbf{z}_i} - \underbrace{\log_2 \frac{1}{q_\phi(\mathbf{z}_i|\mathbf{x}_i)}}_{\text{bits back}}$$

If the sender uses $q_\phi(\mathbf{z}_i \mid \mathbf{x}_i)$ to sample $\mathbf{z}_i$, we can also think about the **expected** cost of a message:

$$-\mathbb{E}_{\mathbf{z}_i \sim q_\phi(\mathbf{z}_i|\mathbf{x}_i)}\left[\log \frac{p_\theta(\mathbf{z}_i, \mathbf{x}_i)}{q_\phi(\mathbf{z}_i \mid \mathbf{x}_i)}\right] = -\text{ELBO}(q_\phi, p_\theta)$$

So if we wanted to obtain a coding scheme that minimizes the expected length of messages, we would just be **fitting a variational autoencoder**! The moral of this story is that compression and probabilistic modeling are two sides of the same coin.